16/7/26

Exp-1 Basics Networking Commands

AIM :-

To study & execute basic networking commands available in LINUX & windows OS for diagnosing & troubleshooting network issues

REQUIREMENTS :-

Pc with Windows / LINUX OS, Terminal (or) Command prompt, Active network connection

THEORY :-

A Computer network is a collection of interconnected devices that share resources & information. Networks commands are built-in utilities that allow administrators & users to configure, & troubleshoot network connections.

Windows provides Commands such as ipconfig, ping, tracert, netstat, nslookup, arp, & route.

LINUX provides equalants : ifconfig/ip, ping, traceroute, netstat/ss, nslookup/dig, arp, & route/ip route.

Step 4: Create project folder

Create:

PacketSniffer

Inside Create:-

sniffer.py

Step 5: Write the Program

Step 6: Save the Program

Save as

sniffer.py

Step 7: Run Powershell

Search

Cmd

Right Click

Run as Administrator

Step 8: Navigate to folder

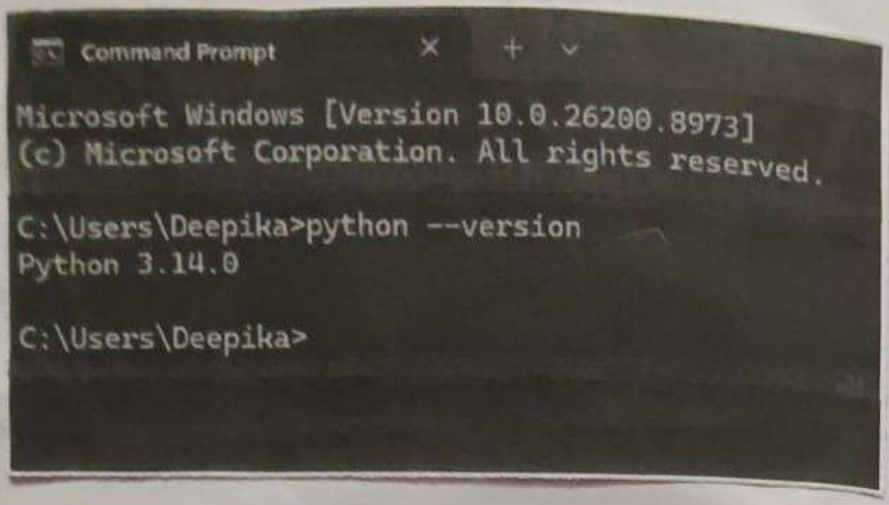
cd desktop

cd packet sniffer

Step 9: Run Program

python sniffer.py

Result:- Thus the Packet Sniffing Using Raw Sockets in Python has been executed & verified



ping.py

```python
import subprocess
import platform
print("=" * 50)
print(" Customized Ping Utility")
print("=" * 50)
host = input("Enter Website or IP Address: ")
system = platform.system()
if system == "Windows":
    command = ["ping", "-n", "4", host]
else:
    command = ["ping", "-c", "4", host]
try:
    result = subprocess.run(command, capture_output=True, text=True)
    print("\nPing Results\n")
    print(result.stdout)
    if result.returncode == 0:
        print("Server Status : Reachable")
    else:
        print("Server Status : Unreachable")
except Exception as e:
    print("Error:", e)
```

```
PS C:\Users\Deepika\Downloads\CustomizedPing> python ping.py
==========================================
 Customized Ping Utility
==========================================
Enter Website or IP Address: google.com

Ping Results


Pinging google.com [142.251.221.174] with 32 bytes of data:
Reply from 142.251.221.174: bytes=32 time=5ms TTL=119
Reply from 142.251.221.174: bytes=32 time=5ms TTL=119
Reply from 142.251.221.174: bytes=32 time=7ms TTL=119
Reply from 142.251.221.174: bytes=32 time=6ms TTL=119

Ping statistics for 142.251.221.174:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 5ms, Maximum = 7ms, Average = 5ms

Server Status : Reachable
```

```
PS C:\Users\Deepika\Downloads\CustomizedPing> python ping.py
==========================================
 Customized Ping Utility
==========================================
Enter Website or IP Address: ajfbjegeufeugwegeg.com

Ping Results

Ping request could not find host ajfbjegeufeugwegeg.com. Please check the name and try again.

Server Status : Unreachable
```

8) Display the ping output, including reply time & packet statistics

9) Check the return code to determine whether the server is reachable

10) Display Reachable or Unreachable based on result



RESULT:

Thus the Customized ping command to test the server connectivity has been executed & verified successfully

Develop a Customised ping command to test the server connectivity

1/08/26 Exp-5

o/p Verified

AIM:

To develop a customized ping command in Python that checks the connectivity between a client and a server by ICMP Echo Request packets & displaying the response time & status

Theory:

A Ping Command is a network diagnostic tool used to determine whether a host (server) is reachable over a network

Working procedure:-

1) Install python & VS Code.
2) Create a python file named ping.py
3) Import the required modules
4) Accept the hostname or IP address from the user
5) Detect the OS
6) Use ping -n 4 on windows or ping -c 4 on Linux/macOS
7) Execute the ping OS command using the subprocess module

```
C:\Users\USER\22000\Downloads\Packet Sniffer>python anonymous_ftp_scanner.py
Enter FTP server address: 127.0.0.1
------------------------------------------------
       ANONYMOUS FTP SCANNER
------------------------------------------------
Target: 127.0.0.1
Port  : 21

[1] Connecting to FTP server...
[-] Anonymous login failed or FTP connection failed.
[!] Error: [WinError 10061] No connection could be made because the target machine actively refused it
```

```
Enter FTP server address: 127.0.0.1
------------------------------------------------
       ANONYMOUS FTP SCANNER
------------------------------------------------
Target: 127.0.0.1
Port  : 21

[1] Connecting to FTP server...
[+] FTP server is reachable
[+] Server response: 220-FileZilla Server 1.12.6
220 Please visit https://filezilla-project.org/

[2] Testing anonymous login...
[+] Anonymous login SUCCESSFUL
[!] The FTP server permits anonymous access.

[3] Checking accessible directory...
[-] Anonymous login failed or FTP connection failed.
[!] Error: 550 No such directory
```

Exp-07 XMLRPC - Calculator

AIM:-

To develop a Simple XMLRPC Calculator to perform arithmetic operations

PROCEDURE:-

1) Verify python installation
2) Create project folder, XMLRPC-calculator wl 2 folders inside server.py & client.py
3) Write Server.py & client.py programs save them inside the created folders
4) Open 2 different cmd prompt & run each folder seperately
5) Start XMLRPC
6) Server.py >> XMLRPC Calculator server running on port 8000.
7) Client.py >> Enter values & derive output

RESULT:-

Hence, the XMLRPC calculator has been successfully implemented & verified

Exp-06 Building an anonymous FTP Scanner Using Python

AIM:-

To develop a python-based anaymous FTP Scanner using the built-in ftplib module & determine whether an FTP Server permits ananymous access

PROCEDURE:-

STEP 1 :- (i) check python installation

STEP 2 :- (ii) Create a file-a python file: anonymous-ftp-Scanner.py

STEP 3 :- (iii) import the ftp lib module

STEP 4 :- (iv) Code the Scanner program

STEP 5 :- (v) Run the program

STEP 6 :- (vi) Get the output

RESULT:-

Thus the building of an anonymous FTP Scanner using python was implemented & executed successfully

Exp-08 Reverse shell using TCP Socket

AIM :-

To develop a program to create reverse shell using TCP Socket

ALGORITHM :-

1) Create the Server.py using the python TCP Server code

2) Also create Client.py to connect with server & for response

3) Run the Server.py in command prompt

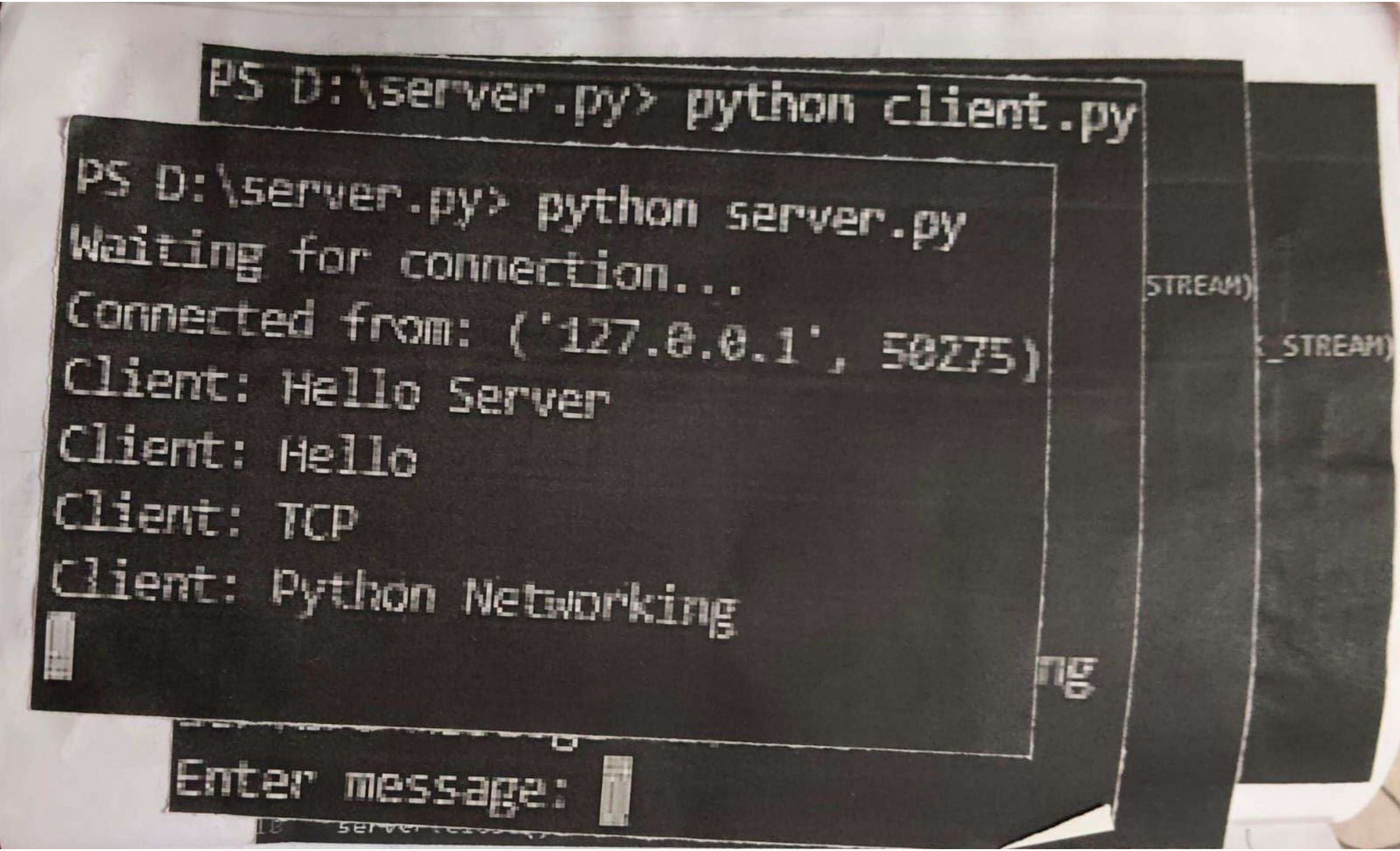
4) Run the Client.py in another new command prompt

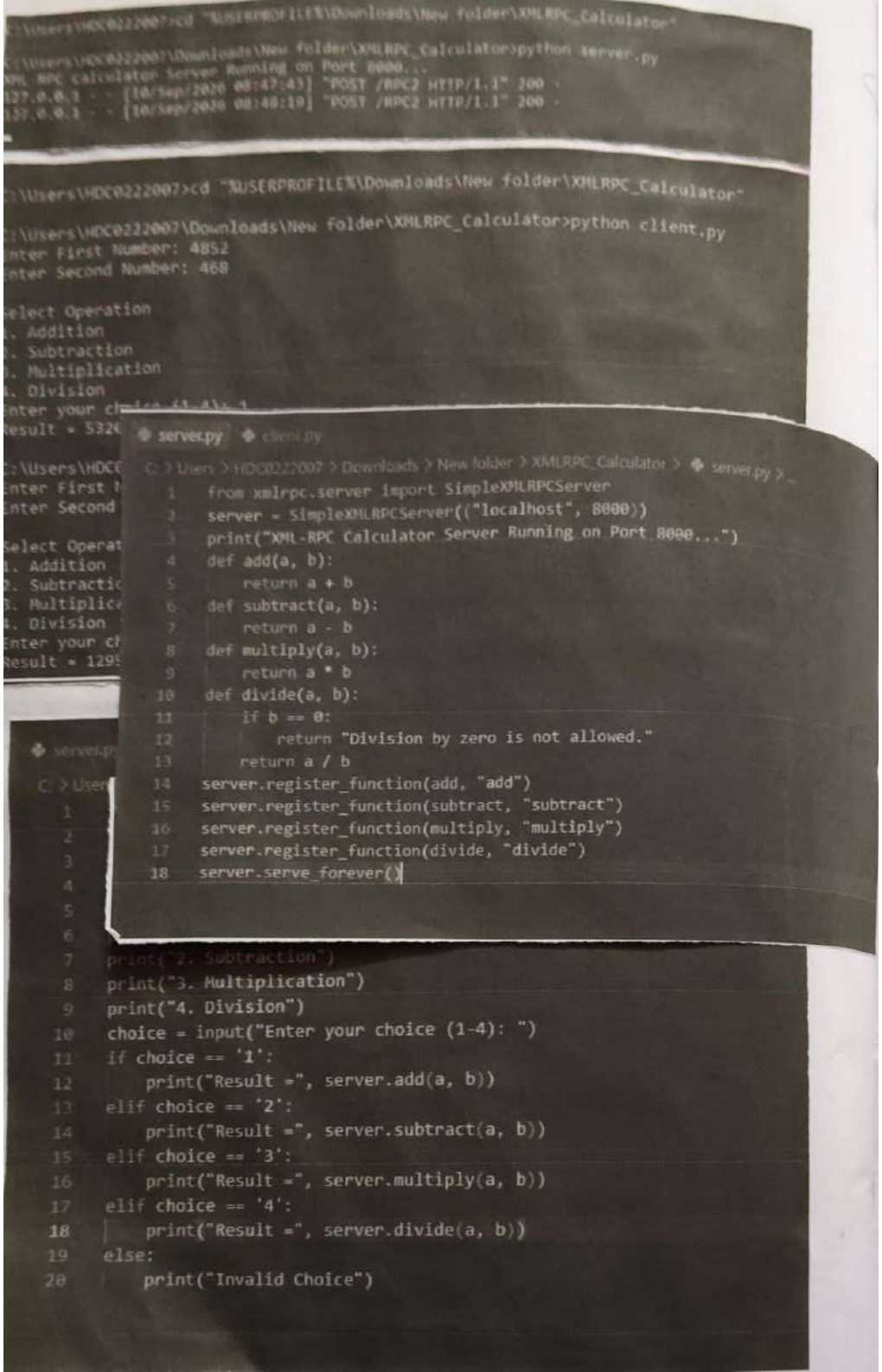
5) Test Multiple Messages

o/p

RESULT :-

Thus the implementation to develop a program to create reverse shell using TCP Socket is executed & verified.


PS D:\server.py> python client.py
PS D:\server.py> python server.py
Waiting for connection...
Connected from: ('127.0.0.1', 50275)
Client: Hello Server
Client: Hello
Client: TCP
Client: Python Networking
Enter message:



```
C:\Users\HDC0222007>cd "%USERPROFILE%\Downloads\New folder\XMLRPC_Calculator"

C:\Users\HDC0222007\Downloads\New folder\XMLRPC_Calculator>python server.py
XML-RPC Calculator Server Running on Port 8000...
127.0.0.1 - - [10/Sep/2026 08:47:43] "POST /RPC2 HTTP/1.1" 200 -
127.0.0.1 - - [10/Sep/2026 08:48:19] "POST /RPC2 HTTP/1.1" 200 -
```

```
C:\Users\HDC0222007>cd "%USERPROFILE%\Downloads\New folder\XMLRPC_Calculator"

C:\Users\HDC0222007\Downloads\New folder\XMLRPC_Calculator>python client.py
Enter First Number: 4852
Enter Second Number: 468

Select Operation
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter your choice (1-4): 1
Result = 5320
```

server.py client.py

C: > Users > HDC0222007 > Downloads > New folder > XMLRPC_Calculator > server.py > ...

```python
from xmlrpc.server import SimpleXMLRPCServer
server = SimpleXMLRPCServer(("localhost", 8000))
print("XML-RPC Calculator Server Running on Port 8000...")
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    if b == 0:
        return "Division by zero is not allowed."
    return a / b
server.register_function(add, "add")
server.register_function(subtract, "subtract")
server.register_function(multiply, "multiply")
server.register_function(divide, "divide")
server.serve_forever()
```

```python
print("2. Subtraction")
print("3. Multiplication")
print("4. Division")
choice = input("Enter your choice (1-4): ")
if choice == '1':
    print("Result =", server.add(a, b))
elif choice == '2':
    print("Result =", server.subtract(a, b))
elif choice == '3':
    print("Result =", server.multiply(a, b))
elif choice == '4':
    print("Result =", server.divide(a, b))
else:
    print("Invalid Choice")
```

Exp-10 Customize switch with network cables modules using Cisco Packet tracer

AIM:

To customize a cisco switch by adding or removing network models using cisco packet tracer & verify the changes in switch interfaces.

Procedure :-

1) Open cisco packet tracer
2) Add a modular switch
3) Open the Physical view
4) Turn off switch
5) Remove an existing module
6) Add a network module
7) Power on switch
8) Verify interface
9) Connect the end devices using 2. copper straigh through wires cables

PC0 → Switch port
PC1 → Switch port

10) Configure IP address for PC0 & PC1 in desktop section

11) Test connectivity & execute in Command prompt ping 190.168.10.3

12) Save the file

Result :-

Thus, the customization of switch with network cables module using cisco packet tracer was implemented Successfully

Exp-09 Design a Simple Topology & Configure One Router, two Switches & PCs Using Cisco Packet tracer

AIM :-

To design & configure a Simple network topology consisting of one Router, two Switches & four PC with Cisco Packet tracer

Procedure :-

1) Open Cisco packet tracer
2) Add devices : 1- Router
   2. Switches
   3. PC's
3) Connect the devices with a copper - straight through cable
4) Open Router CLI & configure Interface Gig0 / Gig0 & save configuration
5) Open PC0 → desktop → Ip configuration then configure Ip address, subnet mask & default gateway
6) Similarly Configure PC1, PC2, PC3
7) Test Connectivity PC0 → PC1

Result :-

Thus the Simple network topology with cisco packet tracer has been implemented & Verified

```
Router# show ip nat statistics

Total active translations: 1 (1 static, 0 dynamic)
Outside interfaces:
  GigabitEthernet0/1
Inside interfaces:
  GigabitEthernet0/0
Hits: 4
Misses: 0
```

```
Router# show ip nat translations

Pro  Inside global   Inside local    Outside local   Outside global
---  200.1.1.100     192.168.1.2     ---             ---
```

Exp-12 Nmap to Discover live hosts in Try Hackme Platform

AIM :-

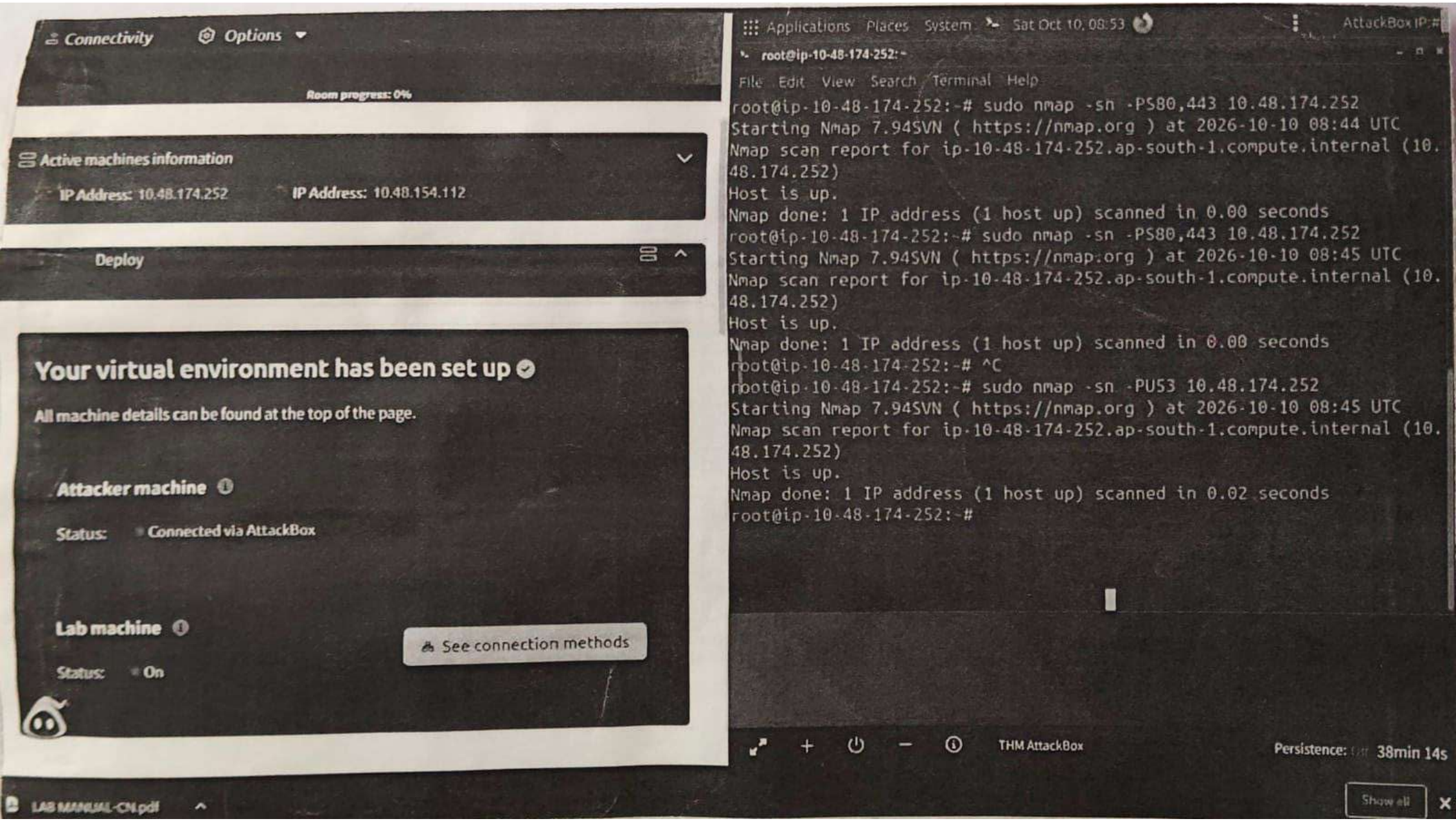
To discover live hosts in a network using NMAP

Procedure :

1) Login to try Hackme
2) Open the required Room
3) Deploy the machine
4) Start the AHackBox
5) Open terminal
6) Verify NMAP Installation
7) Run ARP scan
8) Run icmp scan
9) Perform TCP ping scan
10) Run Tcp ~~ACP~~ ACK ping
11) perform UDP ping
12) Discover all hosts in a network
13) Save the scan results

Result :

Thus live hosts were discovered successfully using NMAP


Connectivity
Options
Room progress: 0%
Active machines information
IP Address: 10.48.174.252
IP Address: 10.48.154.112
Deploy
Your virtual environment has been set up
All machine details can be found at the top of the page.
Attacker machine
Status: Connected via AttackBox
Lab machine
Status: On
See connection methods
Applications Places System Sat Oct 10, 08:53
AttackBox IP:
root@ip-10-48-174-252: ~
File Edit View Search Terminal Help
root@ip-10-48-174-252:~# sudo nmap -sn -PS80,443 10.48.174.252
Starting Nmap 7.94SVN (https://nmap.org) at 2026-10-10 08:44 UTC
Nmap scan report for ip-10-48-174-252.ap-south-1.compute.internal (10.48.174.252)
Host is up.
Nmap done: 1 IP address (1 host up) scanned in 0.00 seconds
root@ip-10-48-174-252:~# sudo nmap -sn -PS80,443 10.48.174.252
Starting Nmap 7.94SVN (https://nmap.org) at 2026-10-10 08:45 UTC
Nmap scan report for ip-10-48-174-252.ap-south-1.compute.internal (10.48.174.252)
Host is up.
Nmap done: 1 IP address (1 host up) scanned in 0.00 seconds
root@ip-10-48-174-252:~# ^C
root@ip-10-48-174-252:~# sudo nmap -sn -PU53 10.48.174.252
Starting Nmap 7.94SVN (https://nmap.org) at 2026-10-10 08:45 UTC
Nmap scan report for ip-10-48-174-252.ap-south-1.compute.internal (10.48.174.252)
Host is up.
Nmap done: 1 IP address (1 host up) scanned in 0.02 seconds
root@ip-10-48-174-252:~#
THM AttackBox
Persistence:
38min 14s
LAB MANUAL-CN.pdf
Show all

Exp: 11 NAT using cisco packet

AIM:

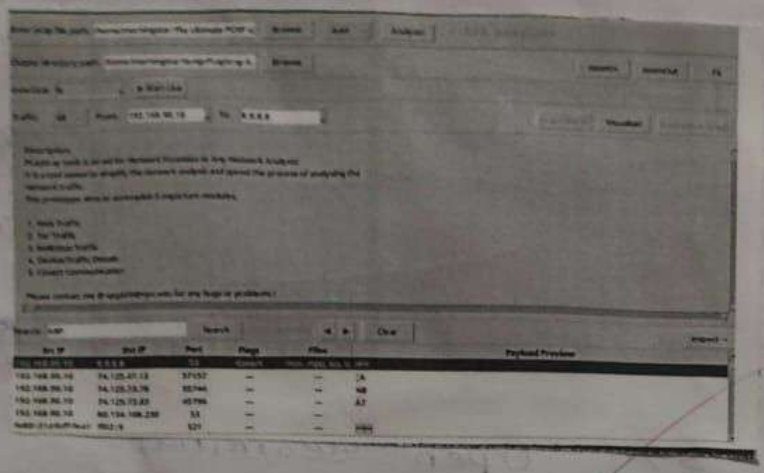
To configure & examine NAT using cisco packet tracer & Verify communication b/w a private network

PROCEDURE:-

1) Open cisco packet tracer
2) Add network devices
3) Connect devices
4) Open Router CLI
5) Configure the Inside Interface
6) Configure the Outside Interface
7) Configure static NAT
8) Save the config
9) Assign Ip address
10) Test Connectivity
11) Verify NAT Translation
12) View NAT statistics

RESULT

Thus NAT was configured & Verified successfully



Exp: 14 Network traffic analysis using wireshark

AIM:

To capture & analyze network traffic using wireshark

PROCEDURE

1) Install & open wireshark
2) Obtain a sample ~~pcap file~~ active network interface
3) Start capturing packets
4) Generate network traffic
5) Apply suitable display filters.
6) Analyze TCP, UDP, IP, ARP
7) Stop & save packet capture

RESULT:

Thus network packets were captured & analyzed successfully using wireshark.

Exp 13 Demonstrate network forensics using pcap x ray tool

Aim

To analyse network traffic using the pcap x-ray tool

Procedure

1) Open the p-cap x-ray tool in a browser
2) Obtain a sample pcap file
3) Upload the pcap file
4) view the analysis dashboard
5) Examine network problems & IP addresses
6) Analyze communication patterns & suspicious activities
7) Generate & save the report

Result :-

Thus network traffic was analysed successfully using pcap x-ray

Exp:15 Web server Analysis using webaliger

AIM :

To analyze web server log files using webaliger

PROCEDURE :

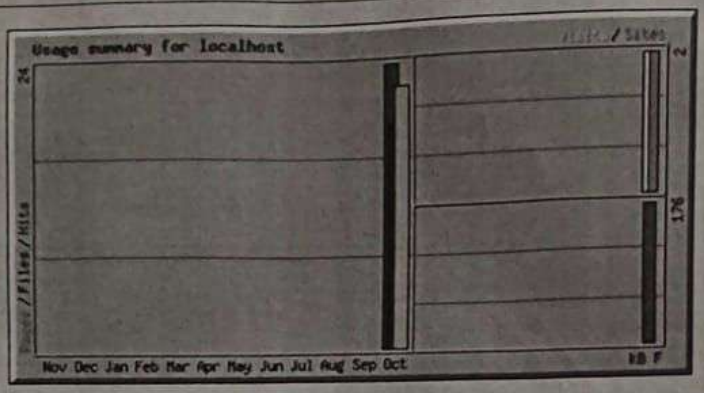
1) Open the ubuntu terminal
2) Install Apache web server
3) Install the webaliger tool
4) Generate web traffic by visiting the website
5) Configure webaliger to analyz the access log
6) Generate & open statistical report
7) Analyze visitor details, hits, page & traffic statistics

Result :-

Thus web server logs were analyzed successfully using weblaliger

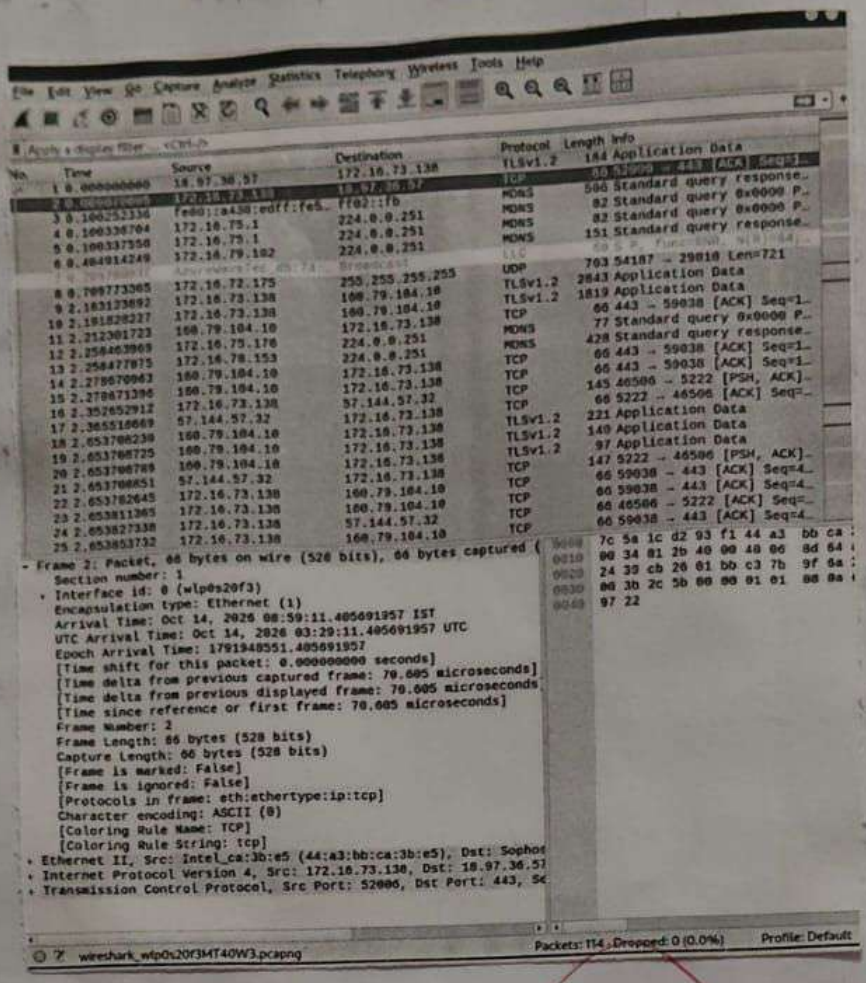
Usage statistics for localhost

Summary by Month
Generated 14-Oct-2026 09:12 IST



Summary by Month										
Month	Daily Avg				Monthly Totals					
	Hits	Files	Pages	Visits	Sites	KB F	Visits	Pages	Files	Hits
Oct 2026	24	19	22	2	2	176	2	22	19	24
Totals						176	2	22	19	24

Generated by Webalizer Version 2.23


File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
1 0.000000000 18.97.36.57 172.16.73.138 TLSv1.2 144 Application Data
2 0.000070005 172.16.73.138 18.97.36.57 TCP 66 59038 → 443 [ACK] Seq=1…
3 0.100252336 fe80::a430:edff:fe5… ff02::fb MDNS 586 Standard query response…
4 0.100336704 172.16.75.1 224.0.0.251 MDNS 82 Standard query 0x0000 P…
5 0.100337558 172.16.75.1 224.0.0.251 MDNS 82 Standard query 0x0000 P…
6 0.484914249 172.16.79.102 224.0.0.251 MDNS 151 Standard query response…
7 0.709773365 172.16.72.175 255.255.255.255 UDP 703 54187 → 29810 Len=721
8 0.709773365 172.16.73.138 160.79.104.10 TLSv1.2 2843 Application Data
9 2.183133892 172.16.73.138 160.79.104.10 TLSv1.2 1819 Application Data
10 2.191828227 160.79.104.10 172.16.73.138 TCP 66 443 → 59038 [ACK] Seq=1…
11 2.212301723 172.16.75.176 224.0.0.251 MDNS 77 Standard query 0x0000 P…
12 2.258463965 172.16.78.153 224.0.0.251 MDNS 428 Standard query response…
13 2.258477875 160.79.104.10 172.16.73.138 TCP 66 443 → 59038 [ACK] Seq=1…
14 2.278670963 160.79.104.10 172.16.73.138 TCP 66 443 → 59038 [ACK] Seq=1…
15 2.278671396 172.16.73.138 57.144.57.32 TCP 145 46506 → 5222 [PSH, ACK]…
16 2.352652912 57.144.57.32 172.16.73.138 TCP 66 5222 → 46506 [ACK] Seq=…
17 2.365516669 160.79.104.10 172.16.73.138 TLSv1.2 221 Application Data
18 2.653708239 160.79.104.10 172.16.73.138 TLSv1.2 140 Application Data
19 2.653708725 160.79.104.10 172.16.73.138 TLSv1.2 97 Application Data
20 2.653708789 172.16.73.138 172.16.73.138 TCP 147 5222 → 46506 [PSH, ACK]…
21 2.653708851 57.144.57.32 172.16.73.138 TCP 66 59038 → 443 [ACK] Seq=4…
22 2.653782645 172.16.73.138 160.79.104.10 TCP 66 59038 → 443 [ACK] Seq=4…
23 2.653811365 172.16.73.138 160.79.104.10 TCP 66 46506 → 5222 [ACK] Seq=…
24 2.653827338 172.16.73.138 57.144.57.32 TCP 66 59038 → 443 [ACK] Seq=4…
25 2.653853732 172.16.73.138 160.79.104.10 TCP
Frame 2: Packet, 66 bytes on wire (528 bits), 66 bytes captured (
Section number: 1
Interface id: 0 (wlp0s20f3)
Encapsulation type: Ethernet (1)
Arrival Time: Oct 14, 2026 08:59:11.405691957 IST
UTC Arrival Time: Oct 14, 2026 03:29:11.405691957 UTC
Epoch Arrival Time: 1791948551.405691957
[Time shift for this packet: 0.000000000 seconds]
[Time delta from previous captured frame: 70.605 microseconds]
[Time delta from previous displayed frame: 70.605 microseconds]
[Time since reference or first frame: 70.605 microseconds]
Frame Number: 2
Frame Length: 66 bytes (528 bits)
Capture Length: 66 bytes (528 bits)
[Frame is marked: False]
[Frame is ignored: False]
[Protocols in frame: eth:ethertype:ip:tcp]
Character encoding: ASCII (0)
[Coloring Rule Name: TCP]
[Coloring Rule String: tcp]
Ethernet II, Src: Intel_ca:3b:e5 (44:a3:bb:ca:3b:e5), Dst: Sophos
Internet Protocol Version 4, Src: 172.16.73.138, Dst: 18.97.36.57
Transmission Control Protocol, Src Port: 52006, Dst Port: 443, Se
0000 7c 5a 1c d2 93 f1 44 a3 bb ca
0010 00 34 81 2b 40 00 40 06 8d 64
0020 24 39 cb 26 01 bb c3 7b 9f 6a
0030 00 3b 2c 5b 00 00 01 01 08 0a
0040 97 22
wireshark_wlp0s20f3MT40W3.pcapng
Packets: 114 · Dropped: 0 (0.0%)
Profile: Default

WINDOWS COMMANDS :-

1) Command : ipconfig
displays : IP address
Subnet Mask
Default gateway

2) Command : ipconfig / all
Displays : Mac address
DHCP Server
DNS Server
Lease information
Adapter details

3) Command : ping google.com
Displays : checks whether the destination is reachable.

4) Command : tracert google.com
Displays : Displays every router (hop) between your Computer & Hardware

5) Command : nslookup google.com
Displays :- DNS Server , IP address of website

6) Command :- route print
Displays :- Network routes , Gateway, Destination networks

```
Microsoft Windows [Version 10.0.26200.6875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kiran>ipconfig

Windows IP Configuration


Wireless LAN adapter Local Area Connection* 1:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 2:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . :
   IPv6 Address. . . . . . . . . . . : 2401:4900:88e4:3e66:a56f:d613:e6e0:a3e3
   Temporary IPv6 Address. . . . . . : 2401:4900:88e4:3e66:65ad:67e5:4f2d:ec1
   Link-local IPv6 Address . . . . . : fe80::66c4:5499:202:4cc9%6
   IPv4 Address. . . . . . . . . . . : 192.168.1.17
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : fe80::1%6
                                       192.168.1.1

Ethernet adapter Ethernet:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
```

```
C:\Users\kiran>ping google.com

Pinging google.com [2404:6800:4000:1006::8a] with 32 bytes of data:
Reply from 2404:6800:4000:1006::8a: time=28ms
Reply from 2404:6800:4000:1006::8a: time=27ms
Reply from 2404:6800:4000:1006::8a: time=30ms
Reply from 2404:6800:4000:1006::8a: time=46ms

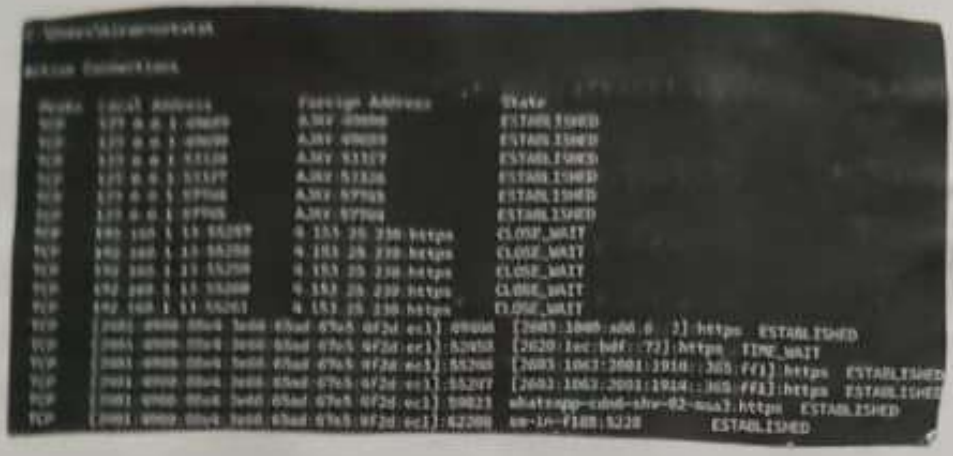
Ping statistics for 2404:6800:4000:1006::8a:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 27ms, Maximum = 46ms, Average = 32ms
```

```
C:\Users\kiran>tracert google.com

Tracing route to google.com [2404:6800:4000:1006::8a]
over a maximum of 30 hops:

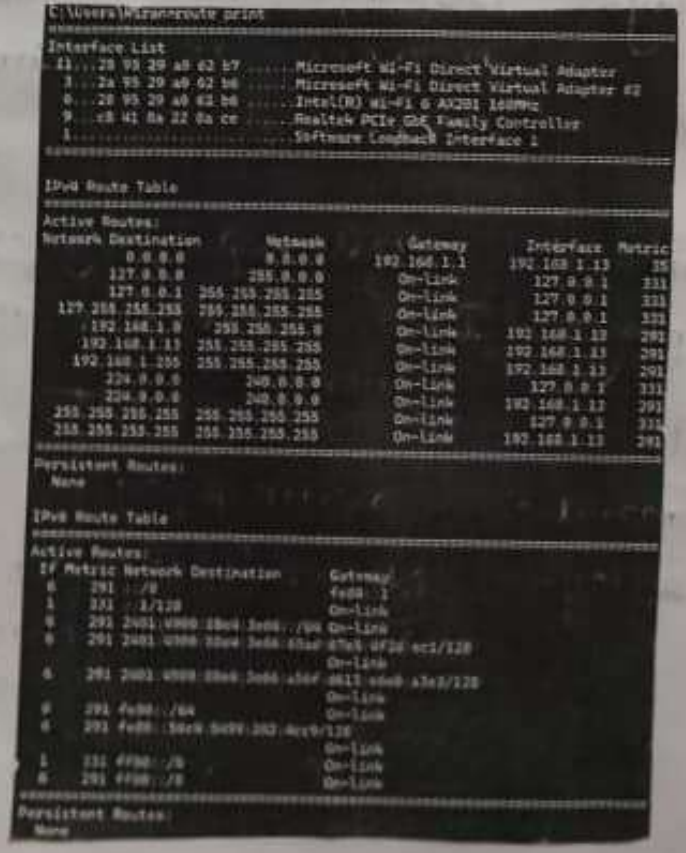
  1     1 ms     1 ms     2 ms  2401:4900:88e4:3e66:5646:17ff:fe21:1f8e
  2     *        *        *     Request timed out.
  3     6 ms     7 ms     6 ms  fc00::9b
  4     4 ms     5 ms     6 ms  2404:a800:3a00:1::9e5
  5     7 ms     *        9 ms  2404:a800::92
  6     8 ms     6 ms     8 ms  2404:6800:8202:2c0::1
  7    10 ms     6 ms     6 ms  2404:6800:8202:2c0::1
  8    10 ms    10 ms    10 ms  2001:4860:0:1::5666
  9     9 ms     6 ms     6 ms  2001:4860:0:1::882a
 10    40 ms    40 ms    38 ms  2001:4860::c:4804:53ca
 11    35 ms    34 ms    34 ms  2001:4860::9:4002:d931
 12    36 ms    35 ms    35 ms  2001:4860:0:1::4e43
 13    47 ms    32 ms    34 ms  2001:4860:0:1::78ce
 14     *        *        *     Request timed out.
 15     *        *        *     Request timed out.
 16     *        *        *     Request timed out.
 17     *        *        *     Request timed out.
 18     *        *        *     Request timed out.
 19     *        *        *     Request timed out.
 20    28 ms    27 ms    28 ms  fx-in-f138.1e100.net [2404:6800:4000:1006::8a]

Trace complete.
```



```
C:\Users\kiran>nslookup google.com
DNS request timed out.
    timeout was 2 seconds.
Server:  UnKnown
Address:  fe80::1

Non-authoritative answer:
Name:    google.com
Addresses:  2404:6800:4000:1010::65
          2404:6800:4000:1010::8a
          2404:6800:4000:1010::8b
          2404:6800:4000:1010::66
          142.251.106.102
          142.251.106.100
          142.251.106.139
          142.251.106.138
          142.251.106.113
          142.251.106.101
```



PROCEDURE :-

1) Open Command prompt / Terminal

2) Execute 'ipconfig' (windows) or 'ifconfig' / 'ip a' (LINUX) to view IP configuration

3) Use ping '8.8.8.8' to test connectivity to Google's DNS server.

4) Execute 'tracert google.com' (windows) (or) 'traceroute google.com' (LINUX) to view packet path

5) Run 'netstat -an' to list active network connections & listening ports.

6) Use 'nslookup google.com' to perform DNS lookup

7) Execute 'arp -a' to view the ARP cache

8) Use 'Route print' (windows) (or) 'route -n' (linux) to view routing table.

9) Records all o/ps & analyse the results.

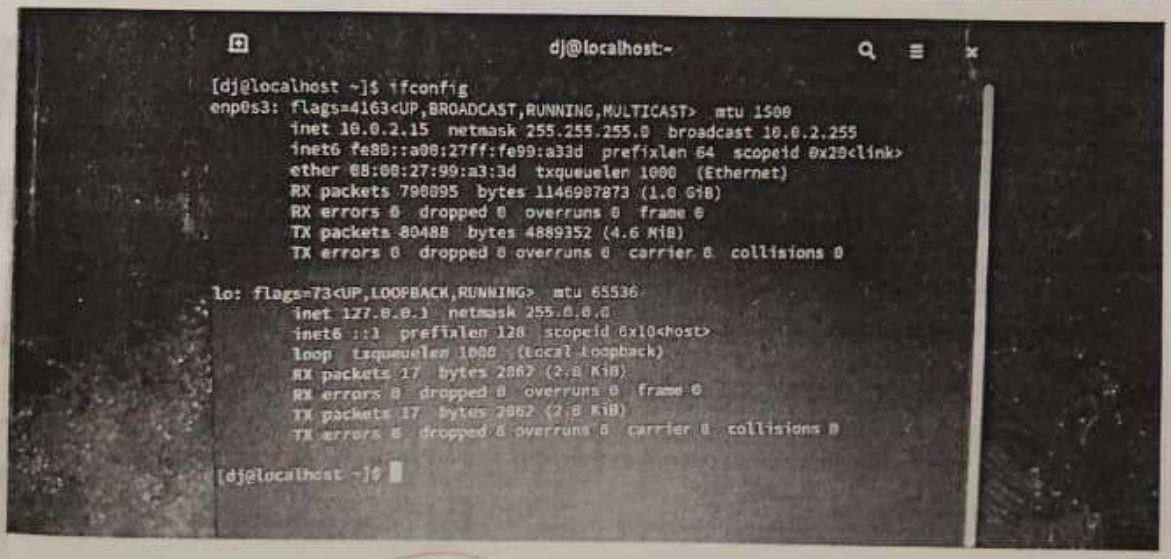
```
C:\Users\kiran>arp -a

Interface: 192.168.1.13 --- 0x6
  Internet Address      Physical Address      Type
  192.168.1.1           54-46-17-21-1f-8e     dynamic
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static
```

```
C:\Users\kiran>hostname
AJAY
```


dj@localhost:~
[dj@localhost ~]$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defaul
t qlen 1000
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo
valid_lft forever preferred_lft forever
inet6 ::1/128 scope host
valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gr
oup default qlen 1000
link/ether 08:00:27:99:a3:3d brd ff:ff:ff:ff:ff:ff
inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
valid_lft 86313sec preferred_lft 86313sec
inet6 fe80::a00:27ff:fe99:a33d/64 scope link noprefixroute
valid_lft forever preferred_lft forever
[dj@localhost ~]$


dj@localhost:~
[dj@localhost ~]$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defaul
t qlen 1000
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo
valid_lft forever preferred_lft forever
inet6 ::1/128 scope host
valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gr
oup default qlen 1000
link/ether 08:00:27:99:a3:3d brd ff:ff:ff:ff:ff:ff
inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
valid_lft 86313sec preferred_lft 86313sec
inet6 fe80::a00:27ff:fe99:a33d/64 scope link noprefixroute
valid_lft forever preferred_lft forever
[dj@localhost ~]$


dj@localhost:~
[dj@localhost ~]$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 10.0.2.15 netmask 255.255.255.0 broadcast 10.0.2.255
inet6 fe80::a00:27ff:fe99:a33d prefixlen 64 scopeid 0x20<link>
ether 08:00:27:99:a3:3d txqueuelen 1000 (Ethernet)
RX packets 790895 bytes 1146987873 (1.0 GiB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 80488 bytes 4889352 (4.6 MiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
inet6 ::1 prefixlen 128 scopeid 0x10<host>
loop txqueuelen 1000 (Local Loopback)
RX packets 17 bytes 2062 (2.0 KiB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 17 bytes 2062 (2.0 KiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
[dj@localhost ~]$

7) Command: ip link
displays: displays MAC address

8) Command: cat / etc / resolv.conf
displays: DNS configuration

Result:

thus the basic networking commands available in LINUX & windows has been executed & verified successfully

7) Command: netstat
Displays: Active TCP connection, Listening ports

8) Command: hostname
Displays: displays Computer name

LINUX COMMANDS :-

1) Command: ifconfig
Displays: Displays ip address

2) Command: hostname
Displays: Displays system hostname

3) Command: ip route
Displays: Shows default gateway. Routing table

4) Command: ping google.com
Displays: ~~trace~~ checks destination is reachable

5) Command: net stat -a
Displays: network statistics

6) Command: arp -a
Displays: displays ARP-table

STP	Category 6·6a	10Gbps	Advantages • Shielded • faster than UTP • Less susceptible to noise & interference Disadvantages • Expensive • Greater installation effort	Gigabit ethernet, 10G Ethernet (55m) widely used in data centres
SSTP	Category 7	10Gbps		Gigabit ethernet 10G ethernet (100m)
Coaxial cable	RG-6 RG-59 RG-11	10-100 Mbps	Advantages • High bandwidth • Immune to interference • Low loss bandwidth • Versatile Disadvantage • Limited distance • Cost • Size is bulky	Speed of signal is 500m Television network High speed internet connection
fibre optic cable	Single mode Multi-mode	100Gbps	Advantages • High speed • High bandwidth Disadvantage • Expensive • Requires skilled installer	Maximum distance of fibre optic cable is around 100 m

Result :-

Thus the study of different types of network cables has been verified.

Learning & Assignment of IP Address Manually to Computers

Exp-3

AIM:

To Understand the IP addressing scheme & manually add IP addresses to Computers on network

THEORY:

1) An IP (Internet Protocol) address is an unique numerical label assigned to each device Connected to a Computer network

2) IPv4 addresses are 32-bit nos written in decimal notations

3) IPv6 uses 128-bit address

4) Subnet mask determines the network & host portions

5) Manual (static) IP assignment is used for Server & network devices requiring a permanent address

6) DHCP assigns IP dynamically

23/10/24

Exp-2 Types of network Cables

Aim :-

Study of different types of network cables.

(a) Understand different types of network cables :-

Different types of network cables are :

(i) Unshielded Twisted Pair (UTP) cable

(ii) Shielded Twisted Pair (STP) cable

(iii) Coaxial Cable

(iv) Fibre optic cable

Cable type	Category	Maximum data Transmission	Advantages / Disadvantages	Application / Use
UTP	Category 3	10 bps	• Advantages → Cheaper in cost → Easy to install as they have a smaller overall diameter • Disadvantage → More prone to CEMI)	10 Base-T- Ethernet
	Category 5	upto 100 Mbps		fast ethernet, Gigabit ethernet
	category 5e	1 Gbps		Fast ethernet, gigabit ethernet

PROCEDURE :

1) In windows : open network & Sharing center
→ change Adapter settings → right click local area Connection → Properties → IPv4 → Set IP manually

2) Enter IP Address : 192.168.1.10. Subnet Mask : 255.255.255.0, Default gateway : 192.168.1.1

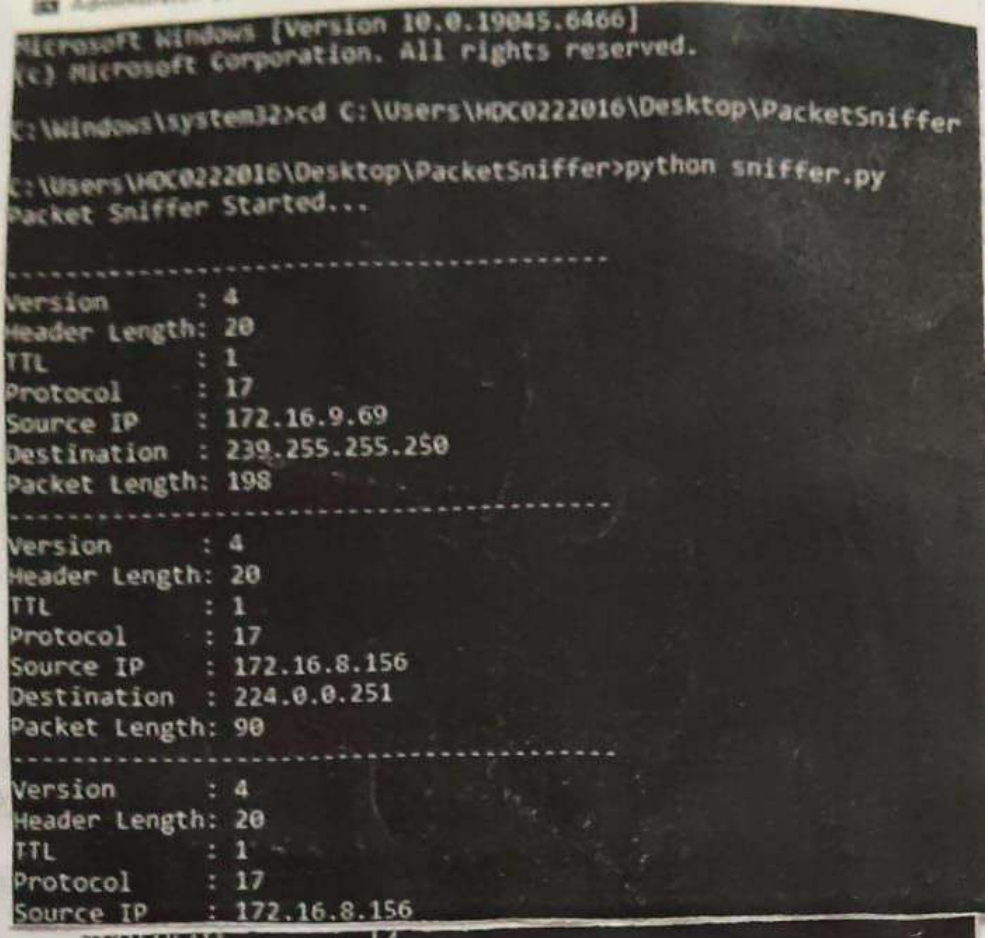
3) Enter DNS Server : 8.8.8.8, click OK

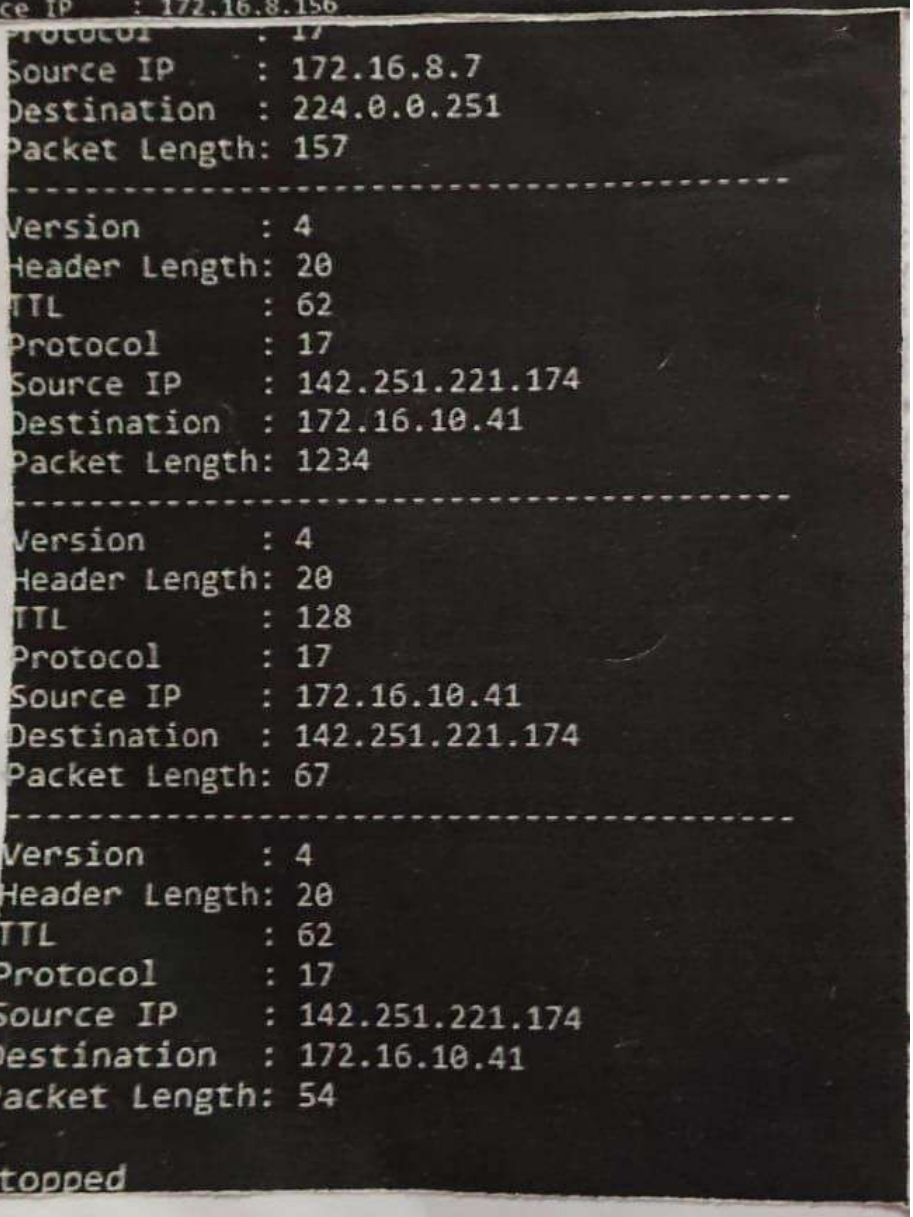
4) In linux use command 'sudo ip add 192.168.1.10/24 dev .

5) It set gateway in linux sudo ip route add default via 192.168.1.1

6) For persist. configuration in Linux, edit / etc / network / interfaces or use

7) Verify the Configuration using ipconfig


Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.
C:\Windows\system32>cd C:\Users\HOC0222016\Desktop\PacketSniffer
C:\Users\HOC0222016\Desktop\PacketSniffer>python sniffer.py
Packet Sniffer Started...
Version : 4
Header Length: 20
TTL : 1
Protocol : 17
Source IP : 172.16.9.69
Destination : 239.255.255.250
Packet Length: 198
Version : 4
Header Length: 20
TTL : 1
Protocol : 17
Source IP : 172.16.8.156
Destination : 224.0.0.251
Packet Length: 90
Version : 4
Header Length: 20
TTL : 1
Protocol : 17
Source IP : 172.16.8.156


Source IP : 172.16.8.7
Destination : 224.0.0.251
Packet Length: 157
Version : 4
Header Length: 20
TTL : 62
Protocol : 17
Source IP : 142.251.221.174
Destination : 172.16.10.41
Packet Length: 1234
Version : 4
Header Length: 20
TTL : 128
Protocol : 17
Source IP : 172.16.10.41
Destination : 142.251.221.174
Packet Length: 67
Version : 4
Header Length: 20
TTL : 62
Protocol : 17
Source IP : 142.251.221.174
Destination : 172.16.10.41
Packet Length: 54
Stopped

```
File Edit Format Run Options Window Help
import socket
import struct

# Create raw socket
sniffer = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP)

# Local machine IP
host = socket.gethostbyname(socket.gethostname())

sniffer.bind((host, 0))

# Include IP headers
sniffer.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)

# Enable promiscuous mode (Windows)
sniffer.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)

print("Packet Sniffer Started...\n")

try:
    while True:
        raw_data, addr = sniffer.recvfrom(65535)

        ip_header = raw_data[0:20]

        iph = struct.unpack('!BBHHHBBH4s4s', ip_header)

        version = iph[0] >> 4
        ihl = iph[0] & 0xF

        ttl = iph[5]
        protocol = iph[6]

        source = socket.inet_ntoa(iph[8])
        destination = socket.inet_ntoa(iph[9])

        print("----------------------------------------")
        print("Version        :", version)
        print("Header Length  :", ihl * 4)
        print("TTL            :", ttl)
        print("Protocol       :", protocol)
```

```
        print("Source IP      :", source)
        print("Destination    :", destination)
        print("Packet Length:", len(raw_data))

except KeyboardInterrupt:
    sniffer.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)
    print("\nStopped")
```

Packets Sniffing Using Raw Sockets in Python

Exp-4

AIM:

To Implement a packet sniffer using raw sockets in Python that captures live network traffic & parses Ethernet, IP, & transport - layer (TCP/UDP/ICMP) headers.

THEORY:-

Raw Sockets : Normal Sockets let an application send / recieve data through the OS's protocol stack, hiding lower-layer headers. A raw socket bypasses this abstraction - it gives the program direct access to the packets at the IP or even the Ethernet layer, including headers.

Procedure :-

Step 1 :- Install Python.

Windows

Visit

https://www.python.org/downloads/

Download latest python

During installation

Add python to PATH
click install Now
Verify installion
Open cmd

Type :-
python --version

Output:
Python 3.12.4

Step 2: Install VS code
Download from
https://code.visualstudio.com/
install normally
install the python extension

Step 3: Verify python
open cmd

type:
python

Output:
>>>

type:
print("Hello")
Output:
Hello

8) Ping the default gateway to Confirm network Connectivity

9) Ping another Computer on the same subnet to verify communication

CODE:

```
sudo ip addr add 192.168.1.10/24 dev eth0
sudo ip route add default via 192.168.1.1
ip a show eth0
ping 192.168.1.1
nmcli con mod 'wired connection' ipv4 addresses
nmcli con mod 'wired connection' ipv4 gateway
nmcli con mod 'wired connection' ipv4 method manual
nmcli con up 'wired Connection 1'
```

RESULT :-

Static IP addresses were successfully assigned to computer. Connectivity was verified using ping Commands b/w hosts on the same subnet.